

Prof. Dr. Andreas Podelski, Dominik Klumpp, Frank Schüssele

Exam for the Lecture
Cyber-Physical Systems – Discrete Models
WS 2020/21

The exam is 12 pages long. There are 9 exercises, listed below.

Solve the exercises on paper. You do not need to print the exam. Scan your solution or take a picture, and upload it to the ILIAS course for the exam as PDF or JPEG. You are responsible for ensuring that your submission is clearly readable, **even if printed in black-and-white**. You may also use a tablet device rather than paper.

You are responsible for making sure your submission is completely uploaded before the deadline. In case you upload multiple files, make sure to upload only one version of the solution for each exercise.

The exam is open-book. Hence, you may use any written materials to help you solve the exercises: lecture slides, exercise sheets, or other available literature. You are NOT allowed to communicate with any other student or other human being about the exam until the exam has officially been terminated and the submission is closed.

You have **90 minutes** to solve the exercises. Additionally, you have **10 minutes** to scan or photograph and upload your solutions.

In case of illness, you must supply proof of your illness by submitting a medical report to the Examination Office. Please note that the medical examination must be done at the latest on the same day of the missed exam. In case of illness while writing the exam please contact the exam organizers, inform them about your illness and immediately see your doctor. The medical certificate must be submitted within 3 days.

A student who disrupts the orderly proceedings of an examination, or attempts to cheat (e.g. by communicating with other students) will **receive a grade of “nicht bestanden” (5.0, fail)** for the written exam. In severe cases, the Board of Examiners will exclude the student from further examinations.

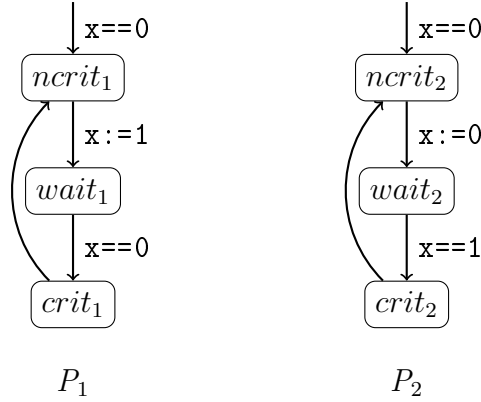
Exercises	Points
1. Mutual Exclusion	of 15
<i>Bonus:</i> exercise (d)	of 2*
2. Finite Automata and Büchi Automata	of 12
3. Transition Systems	of 22
4. Hardware Circuit	of 8
5. Trace Inclusion and Executions	of 8
6. Closure Properties of LT Properties	of 10
7. Invariants, Safety and Liveness Properties	of 16
8. Safety-Liveness Decomposition	of 9
9.* <i>Bonus:</i> Safety and Liveness in Finite Transition Systems	of 6*
Total Points	of 100

Exercise 1

(Mutual Exclusion)

15

Consider the following locking protocol. The initial value of the variable x is 0.



Note: There is a difference between $x:=0$ and $x==0$. The edge labeled with $x:=0$ can always be taken (as there is no guard) and it modifies the value of x . On the other hand, the edge with $x==0$ can only be taken when x has the value 0, and it does not modify the value of x .

- (a) Draw the program graph $P_1 \parallel P_2$, i.e. the program graph for the interleaving of P_1 and P_2 .
- (b) Draw the reachable part of the transition system $\mathcal{T}_{P_1 \parallel P_2}$ for the interleaving of the programs. Use the atomic propositions $\{crit_1, crit_2\}$ that are satisfied, whenever process 1 respectively process 2 are in their critical section.
- (c) Does the protocol satisfy the mutual exclusion property? Explain your answer in sufficient detail.
- (d) Is this a reasonable protocol for parallel programs? Explain your answer in sufficient detail.

2*

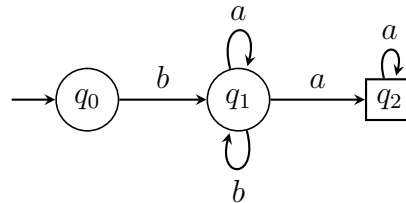
Exercise 2

(Finite Automata and Büchi Automata)

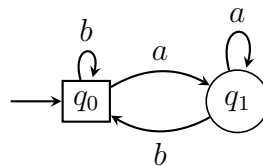
12

In this exercise we consider languages and automata over the alphabet $\Sigma = \{a, b\}$.

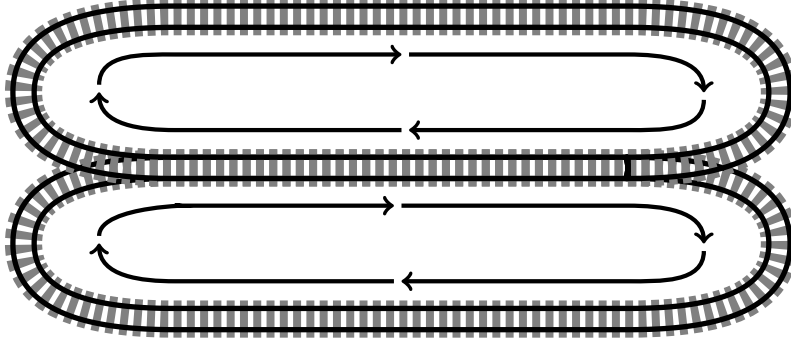
- (a) Draw a Büchi automaton that recognizes the following language:
“The set of all infinite words such that the letter b occurs infinitely often and the letter a occurs only finitely often.”
- (b) Describe the language recognized by the Büchi automaton below in set notation $\{A_1A_2 \dots \in \Sigma^\omega \mid \dots\}$.



- (c) Draw a finite automaton that recognizes the following language:
“The set of all finite words such that every occurrence of the letter b is directly followed by the letter a .”
- (d) Describe the language recognized by the finite automaton below in set notation $\{A_1A_2 \dots A_n \in \Sigma^* \mid \dots\}$.



In this exercise, we consider a cyber-physical system from the railway systems domain. Imagine two trains running on railway tracks, as shown below. As you can see, the tracks for the two trains share a common segment. In order to avoid train crashes, a software controller regulates access to this segment of the tracks. Your task in this exercise is to model this cyber-physical system. It will be made up of three components: the trains ($Train_1$ and $Train_2$) and the controller ($Controller$).



Read all of the following tasks carefully, before you start solving them.

- (a) Draw the transition systems for the trains $Train_1$ and $Train_2$. Include atomic propositions that will allow you to solve sub-exercise (e).

*Each train is initially running on its own segment of the tracks. When approaching the shared segment, the train either receives a message **track_free** from the controller (indicating that the shared track is available), or it receives the message **please_wait** from the controller (indicating that the other train is currently on the shared track). If **please_wait** is received, the train stops and goes into a waiting state. If the **track_free** message is received – either immediately (when approaching) or while the train is waiting –, the train enters the shared segment. When the train arrives on the other end of the shared segment, it notifies the controller by sending **leave_track**, and returns to its initial state (running on its own track).*

- (b) Draw the interleaving of the train transition systems, $Train_1 \parallel Train_2$.

For two of the transitions in this interleaving, justify why they must exist using the SOS-rules for pure interleaving.

- (c) Draw the transition system $Controller$ for the controller.

*The controller keeps track if the shared segment is currently free (which it initially is), or occupied by one of the trains. If the shared segment is free, the controller sends the message **track_free** to approaching trains and marks the segment as occupied. If the shared segment is occupied, the controller sends the message **please_wait** to approaching trains, and the shared segment remains occupied. When the shared segment is occupied and the controller receives the message **leave_track**, the controller marks the shared segment as free again.*

- (d) Draw the entire cyber-physical system, i.e, draw the parallel composition $Controller \parallel (Train_1 \parallel Train_2)$ of the controller with the interleaving from (b).

For two of the transitions in this interleaving, justify why they must exist using the SOS-rules for parallel composition.

- (e) An important property for this cyber-physical system is the property

“The two trains are never both on the shared segment at the same time.”

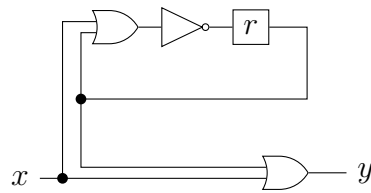
Formalize this property in set notation, using the atomic propositions you introduced in (a).

Does the cyber-physical system satisfy this property? Explain your answer.

Exercise 4

(Hardware Circuit)

8



Provide the labeled transition system of this hardware circuit (i.e., states are labeled by sets of atomic propositions, transitions are not labeled). The states are the evaluations of the input x and the register r . The transitions represent the stepwise behavior of the circuit. The values of the input x change nondeterministically. The atomic propositions $\{X, Y, R\}$ stand for $x = 1$, $y = 1$ and $r = 1$, respectively. Initially the register r has the value 0 (**false**).

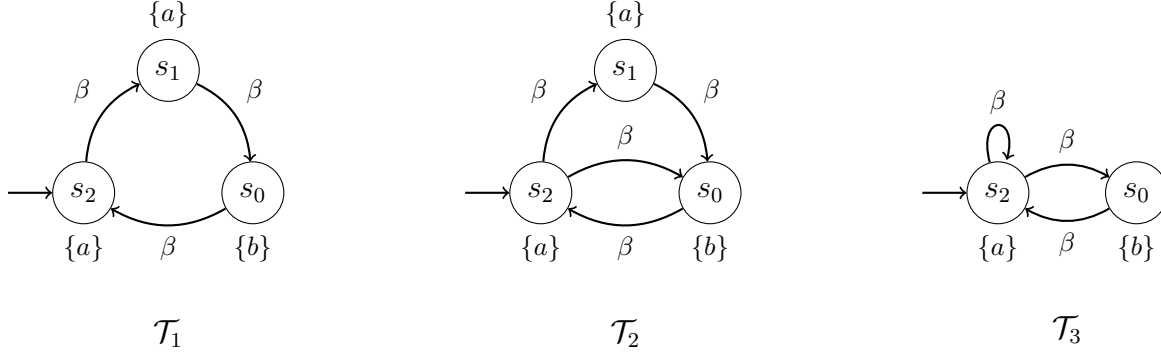
For your reference: $\square \triangleright =$ AND gate, $\sqcup \triangleright =$ OR gate, $\triangleright \circ =$ NOT gate

Exercise 5

(Trace Inclusion and Executions)

8

Consider the following transition systems.



Answer the following questions:

- (a) Does the trace inclusion $Traces(\mathcal{T}_2) \subseteq Traces(\mathcal{T}_1)$ hold?
 If the inclusion holds, explain why this is the case.
 If the inclusion does not hold, give a trace $\tau \in Traces(\mathcal{T}_2) \setminus Traces(\mathcal{T}_1)$.
- (b) Does the trace inclusion $Traces(\mathcal{T}_1) \subseteq Traces(\mathcal{T}_2)$ hold?
 If the inclusion holds, explain why this is the case.
 If the inclusion does not hold, give a trace $\tau \in Traces(\mathcal{T}_1) \setminus Traces(\mathcal{T}_2)$.
- (c) Does the trace inclusion $Traces(\mathcal{T}_1) \subseteq Traces(\mathcal{T}_3)$ hold?
 If the inclusion holds, explain why this is the case.
 If the inclusion does not hold, give a trace $\tau \in Traces(\mathcal{T}_1) \setminus Traces(\mathcal{T}_3)$.
- (d) Give an execution that occurs in one, and only one, of the three transition systems above.

Exercise 6

(Closure Properties of LT Properties)

10

Let AP be a set of atomic propositions. Further, let L, L' be liveness properties over AP , and let S, S' be safety properties over AP . Prove or disprove the following claims:

- (a) $L \cap L'$ is always a liveness property.
- (b) $L \cup S$ is always a liveness property.
- (c) $S \cap S'$ is always a safety property.

Exercise 7

(Invariants, Safety and Liveness Properties)

16

Let $AP = \{c, a\}$ be a set of atomic propositions. Consider the following LT-properties over these atomic propositions:

- (P_1) It is never the case that a holds.
- (P_2) At infinitely many points in time, $\neg c$ holds.
- (P_3) c eventually holds, and a is always false.
- (P_4) If c holds at every point in time, then a also holds at every point in time.

For each of these properties, perform the following tasks:

- (a) Formalize the property in set notation.
- (b) Determine if the property is an invariant. If so, give the invariant condition.
- (c) Determine if the property is a safety property. Justify your answer.
- (d) Determine if the property is a liveness property. Justify your answer.

Exercise 8

(Safety-Liveness Decomposition)

9

Consider the following LT-properties over the set of atomic propositions $AP = \{c, a\}$. For each property P_i , find a safety property P_i^{safe} and a liveness property P_i^{live} such that $P_i = P_i^{\text{safe}} \cap P_i^{\text{live}}$. Describe P_i^{safe} and P_i^{live} in set notation.

You do not need to justify your solution.

$$P_1 = \{ A_1 A_2 A_3 \dots \mid \exists i \in \mathbb{N}. c \in A_i \}$$

$$P_2 = \{ A_1 A_2 A_3 \dots \mid |\{i \in \mathbb{N} \mid a \in A_i\}| = 1 \}$$

$$P_3 = \{ A_1 A_2 A_3 \dots \mid (\exists i \in \mathbb{N}. c \notin A_i) \wedge (\forall i \in \mathbb{N}. c \in A_i \rightarrow a \in A_i) \}$$

$$P_4 = \{ A_1 A_2 A_3 \dots \mid \exists i \in \mathbb{N}. a \in A_i \wedge (\forall j \in \mathbb{N}. j < i \rightarrow c \in A_j) \}$$

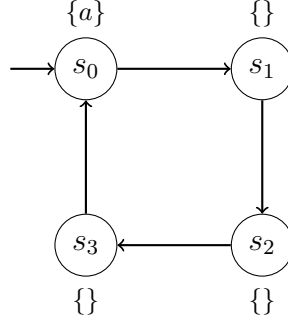
Exercise 9

(Safety and Liveness in Finite Transition Systems)

6*

This exercise deals with the satisfaction relation $(\mathcal{T} \models P)$ with respect to safety and liveness properties.

- (a) Let $AP = \{a, b\}$ be a set of atomic propositions. Consider the following transition system $\mathcal{T}$:



This system clearly satisfies the liveness property “*infinitely often, a holds*”, i.e., for the property

$$\begin{aligned} P_{\text{live}} &= \{ A_1 A_2 A_3 \dots \mid \exists i \in \mathbb{N}. a \in A_i \} \\ &= \{ A_1 A_2 A_3 \dots \mid \forall i \in \mathbb{N}. \exists j \in \mathbb{N}. i \leq j \wedge a \in A_j \} \end{aligned}$$

it holds that $\mathcal{T} \models P_{\text{live}}$.

Does there exist a safety property P_{safe} such that $\mathcal{T} \models P_{\text{safe}}$ and $P_{\text{safe}} \subseteq P_{\text{live}}$? If so, give such a safety property. If no such property exists, explain why this is the case.

- (b) Let $\mathcal{T}$ be an arbitrary transition system with atomic propositions AP , such that $\mathcal{T}$ has only finitely many states. Further, let P_{live} be an arbitrary liveness property over AP , such that $\mathcal{T} \models P_{\text{live}}$.

Does there always exist a safety property P_{safe} such that $\mathcal{T} \models P_{\text{safe}}$ and $P_{\text{safe}} \subseteq P_{\text{live}}$? If this is the case, give the safety property P_{safe} . If such a safety property does not always exist, give a counterexample.

